

Bases de Datos

Clase 24: Búsqueda de Texto — Índices Invertidos y TF-IDF

Lunes 1 de junio

El problema de hoy

Queremos hacer **búsqueda de texto**: encontrar los documentos de una tabla que mencionan ciertas palabras.

"Dame todos los artículos que hablan de **machine learning**."

Primer intento — lo que un programador (o un LLM) escribe sin pensar:

```
SELECT * FROM Articulos WHERE contenido LIKE '%machine learning%';
```

Sintaxis trivial. ¿Cuál es el problema?

Por qué **LIKE** no escala

```
SELECT * FROM Articulos WHERE contenido LIKE '%machine learning%';
```

- **Articulos** tiene 10 millones de filas, **contenido** pesa gigabytes.
- Es un **sequential scan**: leer cada fila completa, comparar el patrón. Lento.
- Un B+ tree sobre **contenido** **no ayuda**: los **%** al inicio impiden saltar a un prefijo conocido. Y aunque pudieramos, el B+ tree indexa el **valor completo del atributo**, no las palabras que contiene.

Para hacer búsqueda de texto rápida, necesitamos un índice **distinto** — uno que entienda que un documento es una **bolsa de palabras**, no un string opaco.

La idea: invertir la pregunta

En vez de recorrer documento por documento buscando la consulta, **invertimos**:

Para cada palabra del corpus, mantenemos la lista de documentos donde aparece.

```
machine    → [doc3, doc7, doc42, doc108, ...] → [3, 7, 42, 108, ...]  
learning   → [doc3, doc12, doc42, doc77, ...] → [3, 12, 42, 77, ...]
```

Buscar "machine learning" se vuelve **interseccionar dos listas** — sin tocar el texto original.

Es la estructura central detrás de **Google, Elasticsearch, Lucene, y tsvector** de **PostgreSQL** para responder consultas de texto libre en milisegundos.

Conexión con índices (Semana 7)

Un **índice invertido** es un índice — al estilo de los B+ tree que ya estudiamos. Cambia **qué se indexa**.

	B+ Tree clásico	Índice invertido
Llave	valor de un atributo	término (palabra)
Entrada	puntero a tupla	posting list (lista de documentos)
Pregunta típica	"¿qué tuplas tienen <code>año = 2020</code> ?"	"¿qué documentos contienen <i>bases de datos</i> ?"
Trade-off	pagar al insertar para acelerar lectura	el mismo

Agenda

1. **Índices invertidos:** estructura y construcción.
2. **Búsqueda booleana** (AND / OR / NOT).
3. **Ranking** y el problema de "¿qué documento es más relevante?"
4. **TF-IDF:** cálculo paso a paso con un ejemplo.
5. Limitaciones y qué vino después (BM25, embeddings).

Índices invertidos

Estructura básica

Diccionario de términos		Posting lists
chile	→	[1, 3, 8, 42, ...]
bases	→	[2, 7, 8, ...]
datos	→	[2, 7, 8, 11, ...]
santiago	→	[1, 3, 15, ...]
...		

- **Diccionario:** estructura ordenada (tipo B+ tree o hash) de los términos del corpus.
- **Posting list:** lista de documentos donde aparece cada término. Suele estar **ordenada por doc-id** para hacer intersecciones eficientes.

Variantes de posting list

Variante	Contenido por entrada	Para qué sirve
Doc-IDs	<code>doc_id</code>	búsqueda booleana
Doc-IDs + frecuencia	<code>(doc_id, tf)</code>	ranking con TF-IDF
Doc-IDs + posiciones	<code>(doc_id, [pos_1, pos_2, ...])</code>	búsqueda de frases (<code>"machine learning"</code>)

En general, mientras más rica la posting list, más rápida y precisa la búsqueda — y más espacio en disco.

Pipeline de construcción

Antes de indexar, procesamos el documento. La estructura es **primero partir, después colapsar**:

```
"Las bases de datos son fundamentales"  
  ↓ tokenización          (partir: texto → palabras)  
  → [Las, bases, de, datos, son, fundamentales]  
  ↓ normalización        (lowercase, quitar acentos)  
  → [las, bases, de, datos, son, fundamentales]  
  ↓ stop words           (descartar ruido)  
  → [bases, datos, fundamentales]  
  ↓ stemming             (colapsar variantes → raíz)  
  → [base, dato, fundamental]  
  ↓  
posting lists
```

Tokenización abre, stemming cierra — ambas para que **bases** y **base** terminen en la **misma** posting list.

Tokenización

Cortar el texto en **tokens** — las unidades léxicas que vamos a indexar. En IR (Information Retrieval) clásico, **un token $\approx$ una palabra**.

Tokenizar es **sensible al idioma**:

- "don't" $\rightarrow$ don't o don + t ?
- "Buenos Aires" $\rightarrow$ ¿uno o dos tokens?
- Chino, japonés: no hay espacios entre palabras.

En PostgreSQL, `to_tsvector('spanish', ...)` aplica un tokenizador específico al idioma.

Tokens IR ≠ tokens LLM

	Tokenización IR	Tokenización LLM
Granularidad	palabra	subléxica (BPE, WordPiece)
Ejemplo	tokenización → tokenización	tokenización → token + iz + ación
Objetivo	unidad indexable	vocabulario compacto para el transformer
Match esperado	consulta humana ↔ palabra del corpus	input ↔ embedding del modelo

Comparten el nombre pero apuntan a problemas distintos. En este curso, "token" significa **palabra**.

Normalización

- **Lowercase:** Chile → chile .
- **Quitar acentos / diacríticos:** año → ano , café → cafe . (Trade-off: pierde precisión, gana recall.)
- **Unicode:** normalización NFC / NFD.

Decisiones de normalización: si normalizamos al **indexar**, debemos normalizar igual al **consultar**. De lo contrario, el match nunca ocurre.

Stop words

Palabras muy frecuentes que aportan poco contenido informativo: el, la, de, que, y, en, ...

- Las eliminamos del índice → ahorramos **espacio** y **tiempo** de consulta.
- Riesgo: consultas como "to be or not to be" quedarían vacías si filtramos todo. Los motores modernos usan listas selectivas.

Stemming

Reducir cada palabra a su raíz.

Palabra	Stem
corredor , correr , corre , corriendo	corr
bases , base	base
databases , database	databas

- Permite que correr matchee con corriendo .
- Algoritmos clásicos: **Porter Stemmer** (inglés), **Snowball** (multi-idioma).
- Es una heurística — produce raíces que no son palabras reales (databas), pero da igual: solo se usan internamente.

Búsqueda booleana

Consultas con AND / OR / NOT

Combinar términos con operadores lógicos.

Consulta	Significado
bases AND datos	documentos que contienen ambos
bases OR fundamentos	al menos uno
bases AND NOT relacional	con bases , sin relacional

Se ejecutan **operando posting lists**:

- AND → intersección de las dos listas.
- OR → unión.
- NOT → complemento contra "todos los doc-IDs".

Intersección eficiente

Si ambas posting lists están **ordenadas por doc-id**:

```
A = [3, 7, 12, 20, 25, 30]
```

```
B = [5, 7, 15, 20, 21, 30]
```

```
pA = pB = 0
```

```
mientras pA < |A| y pB < |B|:
```

```
    si A[pA] = B[pB]: emitir A[pA]; pA++; pB++
```

```
    si A[pA] < B[pB]: pA++
```

```
    si A[pA] > B[pB]: pB++
```

```
resultado: [7, 20, 30]
```

→ Lineal en el tamaño de las listas, **sin saltos aleatorios** en disco. Misma idea que el merge en Sort-Merge Join.

Listas desbalanceadas: 100 vs. 1M

El merge lineal cuesta $O(n + m)$ — domina el m aunque la lista corta tenga 100 entradas. Dos optimizaciones:

- **Skip pointers** cada $\sqrt{m}$ en la lista larga: la búsqueda salta en bloques. Intersección: $O(n \cdot \sqrt{m})$. Es lo que usan Lucene y motores en producción.
- **Gallop / exponential search**: para cada elemento de la corta, búsqueda exponencial (saltos 1, 2, 4, 8, ...) desde el puntero actual. Llega a $O(n \cdot \log(m/n))$ — **óptimo** para el problema.

Con $n = 100, m = 10^6$	Comparaciones
Merge lineal	$\sim 10^6$
Skip pointers $\sqrt{m}$	$\sim 10^5$
Gallop	~ 1.300

Optimización: ¿qué término procesar primero?

Para bases AND datos AND fundamentales :

```
posting(fundamentales) → 120 docs      ← muy pocos
posting(bases)         → 8.000 docs
posting(datos)         → 12.000 docs
```

Empezar por el más corto: intersectar el más raro con el siguiente, etc.

Generalización de un principio que ya conocemos: en planes de consulta, **empujar las selecciones más restrictivas hacia abajo.**

Búsqueda de frases

Para "machine learning" como **frase exacta**, no basta con AND:

```
posting(machine): [d3@5, d7@2, d12@88]    ← término en doc, posición  
posting(learning): [d3@6, d7@40, d12@89]
```

- AND retorna {d3, d7, d12} .
- Pero solo en **d3** y **d12** las palabras aparecen **consecutivas**: posición $5+1=6$ en d3, $88+1=89$ en d12.

→ Si la posting list guarda **posiciones**, podemos resolver frases comparando posiciones tras hacer la intersección de doc-IDs.

En PostgreSQL: `to_tsquery('machine <-> learning')` (operador `<->` = "seguido por").

Ranking

El problema

Una consulta con AND/OR puede retornar **miles** de documentos. ¿Cuál es **más relevante**?

Necesitamos un **score** $s(q, d)$ que ordene los documentos para una consulta q .

Dos intuiciones:

1. Si un término aparece **muchas veces** en un documento, probablemente el documento "trate" sobre ese término.
2. Si un término aparece en **muchos documentos** del corpus, no discrimina — no aporta tanta información.

→ **TF-IDF** captura ambas intuiciones.

TF — Term Frequency

$TF(t, d)$ = número de veces que el término t aparece en el documento d .

Documento	"bases" aparece...	TF(bases, d)
d1 (libro de cocina)	0 veces	0
d2 (artículo de BD)	12 veces	12
d3 (paper de SQL, corto)	4 veces	4

A más TF → más relevante (intuición 1).

Variante: TF normalizado por largo del documento, log-TF ($1 + \log TF$) para suavizar.

IDF — Inverse Document Frequency

$$\text{IDF}(t) = \log \frac{N}{\text{df}(t)}$$

donde N = total de documentos en el corpus y $\text{df}(t)$ = documentos que contienen t .

Término	df	N	IDF
el	1.000.000 (de 1M)	1.000.000	$\log 1 = 0$
bases	8.000	1.000.000	$\log 125 \approx 4.83$
quantum	12	1.000.000	$\log 83.333 \approx 11.34$

A menor df → mayor IDF → término más **discriminativo** (intuición 2).

IDF(el) = 0 no es un bug, es el principio. Una palabra que aparece en *todos* los documentos no aporta nada para decidir cuál es relevante. Su peso en el score es exactamente 0.

TF-IDF

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Score de una consulta $q = \{t_1, t_2, \dots, t_k\}$:

$$s(q, d) = \sum_{t \in q} \text{TF-IDF}(t, d)$$

Por cada término de la consulta, multiplicamos cuánto aparece en el documento por cuán informativo es ese término. Sumamos.

Ejemplo completo

Corpus de **4 documentos** (tras stopwording y stemming simplificados):

d1: "ojo por ojo, diente por diente"	→ [ojo, ojo, diente, diente]
d2: "ojo por ojo, mundo acabará ciego"	→ [ojo, ojo, mundo, acabar, ciego]
d3: "luchas contra mundo, ponte mundo"	→ [luchar, mundo, poner, mundo]
d4: "ojo del huracán, ojo crítico"	→ [ojo, huracan, ojo, critico]

Consulta: $q = \{\text{ojo, mundo}\}$. Calcular $s(q, d_i)$ para cada documento.

Paso 1: TF

Término	TF en d1	TF en d2	TF en d3	TF en d4
ojo	2	2	0	2
mundo	0	1	2	0

Observación: d2 tiene **más** ocurrencias totales (3) que d3 (2). Si rankeamos por **TF puro**, d2 ganaría. Veamos qué hace TF-IDF.

Paso 2: IDF

$N = 4$.

Término	df	IDF = $\log_{10}(N/\text{df})$
ojo	3 (aparece en d1, d2, d4)	$\log_{10}(4/3) \approx 0.125$
mundo	2 (aparece en d2, d3)	$\log_{10}(4/2) \approx 0.301$

mundo es **más raro** que **ojo** en este corpus → su IDF es **más alto** → cada ocurrencia de **mundo** pesa **2.4x** más que una de **ojo**.

Paso 3: TF-IDF y score

$$s(q, d) = \text{TF}(\text{ojo}, d) \cdot 0.125 + \text{TF}(\text{mundo}, d) \cdot 0.301$$

Documento	TF(ojo) · 0.125	TF(mundo) · 0.301	Score
d1	$2 \cdot 0.125 = 0.250$	0	0.250
d2	$2 \cdot 0.125 = 0.250$	$1 \cdot 0.301 = 0.301$	0.551
d3	0	$2 \cdot 0.301 = 0.602$	0.602 ★
d4	$2 \cdot 0.125 = 0.250$	0	0.250

→ **d3 gana**, ¡aunque solo contenga uno de los dos términos! La rareza de **mundo** lo hace **más informativo** que dos menciones de **ojo**. Sin IDF, hubiera ganado d2.

Esta es la lección central: **TF-IDF no premia "más palabras", premia "más palabras raras"**.

Modelo vectorial

Cada documento es un **vector**: una dimensión por término, valor = $TF(t, d) \cdot IDF(t)$.

La consulta también es un vector — un "documento muy corto". Cada término aparece una vez $\rightarrow TF(t, q) = 1 \rightarrow q_t = IDF(t)$.

	ojo	mundo
d1 (TF-IDF) =	[0.250,	0]
d2 (TF-IDF) =	[0.250,	0.301]
d3 (TF-IDF) =	[0,	0.602]
q (TF-IDF) =	[0.125,	0.301]

← TF=1 → q_t = IDF(t)

Cosine similarity

$$\cos(q, d) = \frac{q \cdot d}{\|q\| \cdot \|d\|}$$

Producto punto **normalizado por las normas** — la división por $\|d\|$ controla por largo de documento, evitando que un documento muy largo gane por defecto.

$$\cos(q, d_2) = \frac{0.125 \cdot 0.250 + 0.301 \cdot 0.301}{\sqrt{0.125^2 + 0.301^2} \cdot \sqrt{0.250^2 + 0.301^2}} \approx 0.957$$

⚠ **Score distinto del sum-simple.** El sum-simple $\sum_{t \in q} \text{TF}(t, d) \cdot \text{IDF}(t)$ trataba la consulta como binaria. El modelo vectorial le da TF-IDF también a la consulta y normaliza por $\|q\| \cdot \|d\|$.

TF-IDF como "embedding" léxico

Lo que acabamos de hacer es construir un **vector léxico** del documento — una dimensión por palabra del vocabulario.

En la búsqueda moderna, ese vector lo produce un **modelo de lenguaje** (BERT, OpenAI embeddings):

- El **contenido** del vector cambia: dimensiones latentes, no palabras.
- La **estructura matemática es la misma**: vectores + cosine similarity.

Esa es la base de RAG y búsqueda semántica — material del curso de **Procesamiento de Datos Masivos**.

Limitaciones de TF-IDF y criterios similares

- No entienden **sinónimos** (coche ≠ automóvil).
- No entienden **contexto** ni semántica (banco financiero vs. banco de plaza).
- No capturan **orden** ni proximidad de palabras (a menos que guardemos posiciones).
- Falla con **errores ortográficos** o variantes morfológicas no cubiertas por el stemmer.

La búsqueda moderna combina BM25 (léxico) con **embeddings vectoriales** (semántica densa) — los sistemas se llaman **hybrid search** y suelen mezclar los scores con Reciprocal Rank Fusion. El índice invertido + TF-IDF siguen siendo la **base** sobre la que se construye el ranking inicial; los embeddings agregan la dimensión semántica.

